



NeuroJIT: Improving Just-In-Time Defect Prediction Using Neurophysiological and Empirical Perceptions of Modern Developers

Gichan Lee

Dept. of Computer Science &
Engineering, Hanyang University
Ansan, Republic of Korea
fantasyopy@hanyang.ac.kr

Hansae Ju

Dept. of Applied Artificial
Intelligence, Hanyang University
Ansan, Republic of Korea
sparky@hanyang.ac.kr

Scott Uk-Jin Lee

Dept. of Computer Science &
Engineering, Hanyang University
Ansan, Republic of Korea
scottlee@hanyang.ac.kr

Abstract

Modern developers make new changes based on their understanding of the existing code context and review these changes by analyzing the modified code and its context (i.e., commits). If commits are difficult to comprehend, the likelihood of human errors increases, making it harder for practitioners to identify commits that might introduce unintended defects. Nevertheless, research on predicting defect-inducing commits based on the difficulty of understanding them has been limited. In this study, we present a novel approach NeuroJIT, that leverages the correlation between modern developers' neurophysiological and empirical reactions to different code segments and their code characteristics to find the features that can capture the understandability of each commit. We investigate the understandability features of NeuroJIT in three key aspects: (i) their correlation with defect-inducing risks; (ii) their differences from widely adopted features used to predict these risks; and (iii) whether they can improve the performance of just-in-time defect prediction models. Based on our findings, we conclude that neurophysiological and empirical understandability of commits can be a competitive predictor and provide more actionable guidance from a unique perspective on defect-inducing commits.

CCS Concepts

• **Software and its engineering** → **Maintaining software.**

Keywords

Just-In-Time Defect Prediction, Cognitive Complexity, NeuroSE

ACM Reference Format:

Gichan Lee, Hansae Ju, and Scott Uk-Jin Lee. 2024. NeuroJIT: Improving Just-In-Time Defect Prediction Using Neurophysiological and Empirical Perceptions of Modern Developers. In *39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24)*, October 27–November 1, 2024, Sacramento, CA, USA. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3691620.3695056>



This work is licensed under a Creative Commons Attribution International 4.0 License.
ASE '24, October 27–November 1, 2024, Sacramento, CA, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1248-7/24/10
<https://doi.org/10.1145/3691620.3695056>

1 Introduction

In modern software development processes, the quality of software is determined by the quality of commits, which are incremental software changes. Developers spend most of their time understanding the context of existing code, and they create new commits based on this understanding [39, 60]. Furthermore, to review the quality of continuously submitted commits, developers need to understand the modified code and its context first [17]. This indicates that modern software quality can be heavily influenced by the understandability of commits. If it is difficult to understand commits, developers are more likely to submit defect-inducing commits, and it becomes challenging to detect defect-inducing risks during the review process. For example, an industrial analysis showed that despite passing quality assurance activities, 87% of severe defects in deployed code in the software industry are caused by human cognitive failures [27]. Misunderstandings of commits are reported as the most significant issue preventing the detection of bugs during modern code reviews [5, 16, 18, 53]. Nevertheless, research on predicting defect-inducing commits based on the difficulty of understanding them remains insufficiently explored.

To express the difficulty of understanding source code, related studies have proposed various metrics to capture essential complexity of source code [2–4]. Using these metrics to measure code understandability has practical value, as source code is a common product in most modern software development [22]. However, recent case studies report that most traditional metrics fail to capture the actual difficulties modern developers experience in understanding the code [24, 38, 43]. While human-centric studies have also investigated the relationship between values measured by biometric sensors attached to developers' bodies while modifying source code and software defects [32, 40], predicting defect-inducing commits through biometric sensor measurements is challenging in providing actionable guidance to improve commit quality compared to utilizing source code in practice. Therefore, an alternative approach is needed to reflect the practical perceptions of modern developers while utilizing information that can be extracted from source code.

Based on these research backgrounds, our aim is to utilize the neurophysiological responses elicited in modern developers during the process of understanding source code from NeuroSE studies and the results from large-scale surveys where developers directly assessed the difficulty of understanding code. State-of-the-art NeuroSE research has explored the correlation between various characteristics of code snippets understood during complex cognitive tasks and developers' neurophysiological reactions, such as increased

brain activity and elevated blood oxygen levels in specific brain regions, to better understand how these responses influence software development processes and code comprehension [24, 38, 43]. Additionally, recent large-scale surveys involving a statistically significant number of modern developers have provided new insights into the causes of difficulties in understanding source code [3–5]. Based on these insights, we developed NeuroJIT, a framework that automatically extracts metrics supported by practical evidence of the difficulties in understanding source code, as identified by both NeuroSE research and large-scale empirical studies.

Through the analysis of 27,195 commits from 8 Apache projects, we investigate whether NeuroJIT's features can capture defect-inducing risks, how these features differ from widely adopted features used to capture these risks, and whether NeuroJIT can improve the performance of existing just-in-time defect prediction models. To the best of our knowledge, this study is the first investigation to focus on "how developers did it" in utilizing the understandability of commits for just-in-time defect prediction rather than the previous research focused on "what developers did". Additionally, this study is a collaborative research effort that empirically evaluates the results of state-of-the-art NeuroSE research and large-scale empirical surveys on code comprehension based on a large number of real-world commits [1]. We publicly release NeuroJIT scripts and datasets online to support future just-in-time defect prediction research that leverages commit understandability, providing a rigorous testbed for upcoming related neurophysiological and empirical studies [33].

2 Approach

2.1 Research Questions

The goal of this study is to determine whether features that capture the understandability of commits can help predict defect-inducing commits. We aimed to achieve this goal by developing the NeuroJIT framework, which extracts code features supported by the neurophysiological and empirical evidence. Through experiments using NeuroJIT, we answer the following research questions:

RQ1: Does the understandability of commits have predictive power for defect-inducing commits?

RQ2: Does the understandability of commits provide exclusive information to predict defect-inducing commits?

RQ3: Can the understandability of commits improve the performance of just-in-time defect prediction models?

RQ1 tests the hypothesis that the harder a commit is to understand, the more likely it is to be defect-inducing. To answer RQ1, we perform logistic regression analysis and examine the statistical differences between defective and clean commits, represented by understandability features of NeuroJIT.

RQ2 tests the hypothesis that features capturing the understandability of commits provide different information from the widely-adopted features used to predict defect-inducing commits. To answer RQ2, we compare the prediction results of a model trained only with understandability features to a model trained only with widely-adopted features for predicting defect-inducing commits. Additionally, we qualitatively analyze the prediction results of each model on the same samples to investigate the exclusive information provided by understandability features.

RQ3 tests the hypothesis that information about the understandability of commits can practically improve widely adopted just-in-time defect prediction models. To answer RQ3, we construct just-in-time defect models by combining baseline and understandability features and check for statistically significant performance improvements.

2.2 Commit Understandability Features

To obtain features that capture the understandability of commits, we aimed to aggregate studies that provide neurophysiological and empirical evidence on the features. To this end, we conducted a snowball search focusing on a recent tertiary review that includes 855 source code metrics and 53 literature reviews on modern developers' perceptions [1], and two literature reviews that examined 110 primary studies on neurophysiological reactions (e.g., brain activity, heart rate variability, electrodermal activity, and eye movement) of modern developers while understanding source code [6, 22]. We set the following criteria for aggregation: (i) a minimum number of samples considering statistical significance, (ii) the expertise of experimental participants representing modern developers, (iii) the quality of code snippets used in experiments, (iv) the credibility of publication venues based on Google Scholar metrics, and (v) the publication year to ensure relevance to current software development practices. The authors iteratively replicated the collected studies and discussed the replication results for finding the evidence for the commit understandability features.

Table 1 summarizes each commit understandability feature of this study, along with its definition and the corresponding neurophysiological and empirical evidence. The 10 commit understandability features are derived from various metrics that were identified in multiple NeuroSE studies and large-scale empirical surveys as being correlated with the difficulties modern developers experience in understanding source code. We verified whether each feature is properly estimated in the code snippets used in the experiments of the studies, and whether the reported correlations are accurate. By selecting only the features that passed these verification steps, we aimed to more accurately capture the understandability of commits. In the evidence column of Table 1, the neurophysiological reactions measured across NeuroSE studies show statistically significant correlations with cognitive loads (or the perceived difficulties), varying in magnitude. The empirical evaluations of code understandability by modern developers have consistently validated their statistical significance, supported by a large number of developers involved.

To implement the NeuroJIT framework that automatically measures the features listed in Table 1 for each commit, we developed all metric calculators anew according to the definitions of each metric. Despite the existence of several published metric calculators, we chose to implement new wheels because the available calculators were not designed to extract metrics from commits and showed inconsistencies in defining some features. For example, one traditional metric, HV, was not designed considering specific languages, leading to inconsistent calculations across different calculators with varying target languages. For future verification and replication, the calculation methods for all understandability features are comprehensively detailed in the NeuroJIT replication package [33].

Table 1: Commit Understandability Features of NeuroJIT and Their Evidence

Description	Evidence
HalsteadVolume (HV) The number of data components in code segment [23].	HV significantly correlates with the cognitive loads of 27 developers understanding source code, as measured by the electrical impulses generated by large groups of neurons in the brain [24]. It is also associated with the activation of Brodmann areas, brain regions linked to code comprehension, as measured by fMRI, and with the perceived difficulty of understanding in 19 developers [43].
TermEntropy (TE) The relative distribution of unique terms in the source code (i.e., keywords, identifiers, and operators) [46]. TE increases with the uniform distribution of terms, but decreases when specific terms dominate.	TE has a significant correlation with the results of 120 developers' evaluations of the syntactic understandability of source code [46]. TE also partially represents lexical inconsistencies, which correlate with cognitive loads measured by the duration of eye fixation and blood-oxygen-levels in brain involved in bug localization [19, 20].
DepDegree (DD) The degree of low-level dependencies between program operations in a use-def graph generated by the reaching definitions of variables [8]. DD reflects the extent to which developers need to track information flow.	DD strongly correlates with both the perceived difficulty of 19 developers and the activation of Broca's areas in the brain, which govern syntactic working memory [43]. DD also has a significant correlation with the level of deactivation in the brain's default mode network in 28 developers as they focus more on understanding the source code [44].
MaxDepthNestingLoop (MDNL) The max depth of nesting loop.	A large-scale survey of 392 modern developers reported that 68% of respondents identified deep nesting as a significant hindrance to code understandability [3]. Additionally, in a survey of 100 developers about complexity triggers, deep nesting ranked among the top two triggers overall [4].
NonStructuredBranch (NB) The number of non-structured branch statements (i.e., break and continue).	NB correlates with the activation of the middle frontal gyrus in 19 developers, which is activated when attention and working memory are engaged [43]. NB also correlates the perceived difficulty of loop structures [21].
ExternalCall (EC) The number of external calls (i.e., APIs and library calls).	During the process of predicting the behavior of external code, it was revealed that 46 developers exhibit increased brain activation and mental effort [24, 43]. Additionally, over half of 395 developers perceive that code making or receiving many calls should be prioritized to be inspected for defect-inducing risks [58].
NumberOfParameters (NOP) The number of parameters.	NOP is correlated with the activation of the frontal lobe brain areas in 19 developers, which are associated with language comprehension and perceived difficulty [43].
NumberOfGlobalVariables (NOGV) The number of global variables.	255 modern developers perceive high NOGV is a major trigger that hinders code understanding and complexity of programming tasks [3].
NumberOfMostTerms (NOMT) The number of terms in the line with the most terms.	Many terms and their interactions in a constrained area correlate with the cognitive loads of 19 developers and are recognized as a preattentive indicator of the loads [43]. They also contribute to increased mental effort for 182 developers, as the information cannot be captured as a single unit at a glance [3].
IncorrectIndentations (II) The number of warnings for incorrect indentations examined by <i>Checkstyle</i> [26].	In a survey of 392 developers regarding complexity triggers, 80% of respondents identified II as a major accidental complexity trigger that hinders the structural understanding of code semantics [3, 20].

2.3 Data Preparation

In this study, we employed the ApacheJIT dataset [30] to construct our experimental dataset. ApacheJIT contains commits maintained until 2019 from 13 Apache Java projects. We chose ApacheJIT because it (i) provides access to the original source code of commits from different projects that are currently being maintained, (ii) offers more comprehensive domain coverage than well-known commit datasets [29, 62], and (iii) includes the features proposed by

Kamei et al. [29], which are the most widely adopted in the just-in-time defect prediction research [2]. After selecting the dataset, we followed the basic structure of the software engineering data cleaning process model to remove the following samples and features that could potentially distort model training [35]: (i) samples with incorrect feature values, (ii) samples with missing values for certain features, (iii) duplicate samples, and (iv) features that had the same value across all samples. Meanwhile, we found numerous commits in ApacheJIT that added multiple new files, added or

Table 2: Summarized Statistics of Dataset

Project	# Defective Commits	# Clean Commits	# Changed Methods	# Changed Lines (avg.)	# Context Sizes (avg.)	Fixing Time (avg.)	Duration
ActiveMQ	285 (11.79%)	2132 (88.21%)	2.2	14.4	66.1	475.6	2005-12-14 ~ 2019-12-03
Camel	481 (7.88%)	5622 (92.12%)	3.3	17.5	86.5	831.3	2007-03-19 ~ 2019-12-26
Flink	334 (11.29%)	2625 (88.71%)	3.2	23.5	116.3	232.8	2010-12-18 ~ 2019-12-26
Groovy	519 (16.32%)	2661 (83.68%)	2.5	18.6	85.9	587.3	2003-09-11 ~ 2019-12-25
Cassandra	744 (22.37%)	2582 (77.63%)	2.4	16.3	98.1	320.6	2009-03-23 ~ 2019-12-17
HBase	771 (27.62%)	2020 (72.38%)	2.9	21.1	117.9	242.3	2007-04-07 ~ 2019-12-11
Hive	1011 (45.36%)	1218 (54.64%)	2.5	20.6	131.5	204.9	2008-09-12 ~ 2019-12-25
Ignite	345 (8.23%)	3847 (91.77%)	2.6	17.8	141.2	827.7	2014-02-20 ~ 2019-12-24
	4490 (16.5%)	22707 (83.5%)	2.7	18.7 (17.7%)	105.4	465.3 days	

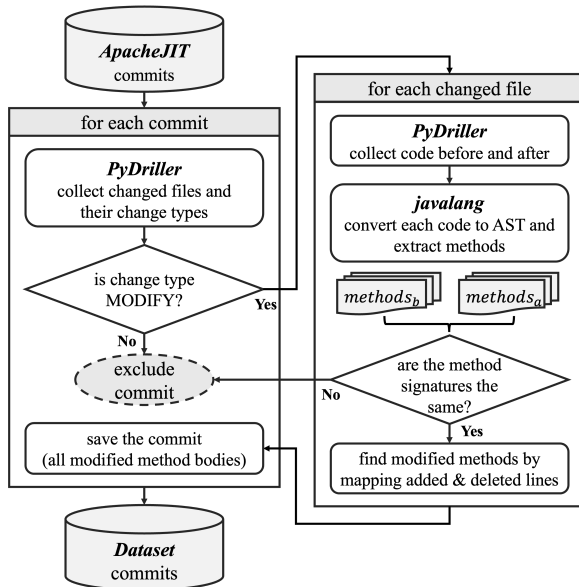
removed entire classes in a single commit, or incorporated entire external libraries into the project. We determined that such commits are infrequent and that developers are unlikely to engage in understanding the existing code context or perform review activities for these changes in their workflow. Therefore, considering the goal of this study, we selected only those commits that modify existing methods. We limited the scope of this study to prevent experimental results from being distorted by overemphasizing what developers need to understand, even if it meant reducing the generalizability of our findings.

Figure 1 details the method and process of selecting commits for the dataset in this study. We used the open-source git repository mining tool *PyDriller* [50] to check whether a commit modifies existing files and collected the pre- and post-commit code. Then, we applied *javalang* [55], a tool for generating Java ASTs, to verify whether the commit modifies existing methods, ensuring that the

commits align with the scope of this study. After extracting the understandability features from the modified code and its context for each method changed by the commits, we used the average value of the feature values calculated for each method as the representative value. We built a dataset of 27,195 commits as shown in Table 2.

Table 2 provides a summary of the commits used in this study. The ratio of defective to clean commits for each project shows a significant imbalance due to the inherent nature of defect-inducing commits. To address this, we devised a resampling plan for the training set to prevent the machine learning algorithm from focusing disproportionately on predicting clean commits. On average, each commit modified about three methods and changed lines corresponding to 20% of the methods it altered. We expected that there would be minimal bias in the dataset caused by unusual commits, as the commits did not show patterns of changing entire methods or too few lines. Meanwhile, one notable column is the Fixing Time (avg.). Fixing time refers to the duration between the detection of a defect in a commit and the time taken to fix it. With the long durations of the projects in our dataset, we observed that the average verification latency was high. Related studies on fair performance evaluation of just-in-time defect prediction models [11, 37, 41, 57] have pointed out that ignoring verification latency can lead to artificially inflated performance metrics. Therefore, to ensure fair performance evaluation, we designed a performance validation method that considers the insights of related studies and the long verification latency observed in our dataset (detailed in section 2.4).

After obtaining the dataset, we conducted a feature selection process. First, to prevent distortions in machine learning algorithm training, we applied the Spearman rank-order correlation test [28] to the baseline feature group, understandability feature group, and combined feature group to check for correlations between features in each group. The Spearman rank-order correlation test outputs 1 for a perfect positive correlation, -1 for a perfect negative correlation, and 0 for no correlation. We used the commonly used threshold values of 0.7 and -0.7 [28] to retain only the feature with the least correlation with other features among pairs of features with strong correlations. Additionally, if three or more features showed strong correlations, we performed feature engineering such as dividing one specific feature by another feature.

**Figure 1: Process of Collecting Commits for Dataset**

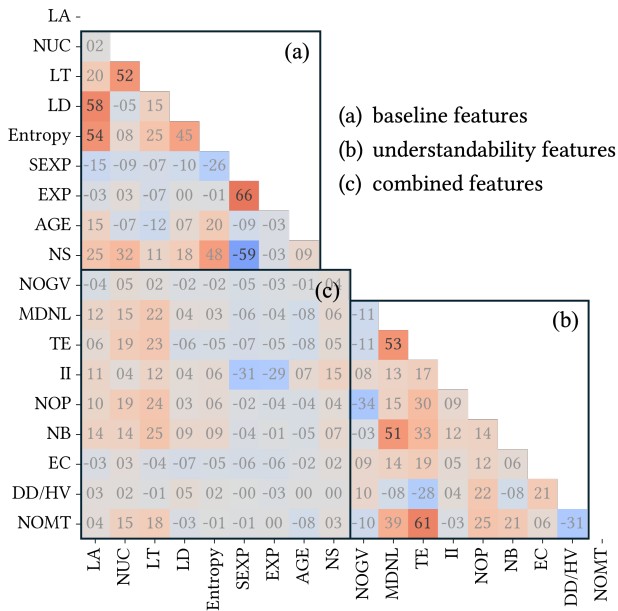


Figure 2: Correlations of Features in Each Group

Figure 2 shows the Spearman correlation coefficients between features in each feature group after feature selection and engineering. The background color of each box approaches red as the correlation coefficient nears 1, and blue as it nears -1. The two-digit numbers in each box represent the decimal places of the correlation coefficient. Among the baseline features, NF and ND were removed not only due to their strong correlation with each other but also because of their strong correlation with Entropy. Additionally, REXP was removed because it showed a near-perfect correlation with EXP. Among the understandability features, DD was divided by HV, and HV was removed due to the strong correlation between DD, HV, and TE. After completing feature selection and engineering within each group, we combined the baseline features and understandability features as shown in Figure 2 (c) to check for correlations. It indicates that there are no strong correlations between the features.

2.4 Just-in-Time Defect Prediction

To build just-in-time defect prediction models, we selected the random forest and XGBoost algorithms [10, 13]. Random forest and XGBoost are ensemble algorithms based on decision trees, known for their robustness against overfitting and multicollinearity through randomness and regularization. Specifically, random forest and XGBoost have been frequently used in the just-in-time defect prediction field due to their strong performance, making them representative choices. Therefore, we aimed to train the random forest and XGBoost algorithms using the implementations in the Python-based *scikit-learn* [42] and *xgboost* [14] packages. To ensure smooth replication and avoid bias introduced by hyperparameter settings, we did not tune hyperparameters during the training process. All models used in this study were trained based on the default hyperparameters provided by the packages, and the original models are available in the replication package as pickles.

For training and evaluating the machine learning algorithms, we adopted the F1-score, False Positive Rate (FPR), AUC, Matthews Correlation Coefficient (MCC), and Brier score. Each performance metric has strengths in handling data imbalance but also has different limitations, which has led to active discussions in related studies about which metrics to use [9, 31, 49, 51, 61]. We chose to use all these metrics to evaluate the experimental results comprehensively, rather than relying on a single metric that might lead to misunderstandings about the reliability of the evaluation results.

The F1-score is a harmonic mean of precision and recall, used to evaluate binary classification models. It ranges from 0 to 1, with values closer to 1 indicating superior performance. The FPR measures the proportion of negative instances incorrectly classified as positive. It ranges from 0 to 1, with lower values indicating better model performance. AUC represents the area under the receiver operating characteristic curve that shows the relationship between true positive and false positive rates. The closer the AUC is to 0.5, the worse the model is at distinguishing between true and false, while the closer it is to 1, the better it can distinguish. The MCC measures the quality of binary classifications, considering true and false positives and negatives. The MCC value varies from -1 to 1, with 1 representing perfect agreement between prediction and reality and -1 representing the opposite. The Brier score evaluates the accuracy of probabilistic predictions, ranging from 0 to 1, with lower values indicating better performance.

Meanwhile, to fairly evaluate the performance of just-in-time defect prediction models, it is essential to consider the characteristics of commits, such as chronology, verification latency, and concept drift. Commits inherently have a temporal order. Therefore, when building the models, it is crucial to avoid data leakage by ensuring that future commits are not used to predict past commits. Additionally, commits often exhibit verification latency, where a commit initially considered clean is later identified as defective when a defect is discovered. Thus, predicting commits at a certain point in time may be biased if the model is trained on commits immediately preceding that point. Lastly, the defect-inducing patterns of commits can change over time, showing concept drifts. This means that the model should be trained on commits that are as close in time as possible to the commits being predicted.

We referred to the techniques used in previous studies [11, 37, 41, 57]. These studies excluded initial and final commits of the projects, where concept drift can be evident (typically based on a period of 3 months), and constructed sliding windows that follow the project timeline for training and evaluating models, taking into account chronology. Additionally, a gap is designed to establish a specific time interval between these sets to accommodate verification latency. Once the window is set, it slides at fixed intervals, training models and measuring performance. However, the studies determine all intervals based on duration, assuming that commits are consistently made throughout each period. If a project's duration is excessively long and variability in development pace occurs, assuming consistent submission rates can lead to significant variances in performance evaluation. Therefore, we aimed to maintain the benefits of the sliding window technique while avoiding bias due to the fixed time interval with long duration. When implementing the sliding windows, we based all intervals not on the duration but on the number of commits contained within a certain period.

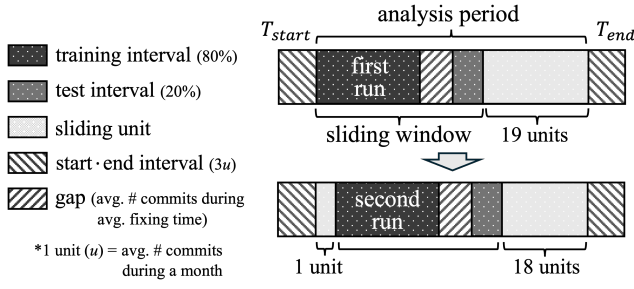


Figure 3: Training and Testing Methods on the Models

Figure 3 depicts the training and testing method of the predictive model designed considering the aforementioned characteristics of commits in detail. We set the average number of commits submitted per month for each project as the basic unit and excluded the initial and final commits of the project by three units from the dataset. The window size for each project was set to accommodate a slide of 20 units, with consideration for concept drift. The gap was determined by the average number of commits generated during the average timeframe for each project to identify defects and submit defect-fixing commits. Excluding the gap within the sliding window, we preserved the sequence of the remaining commits. We divided the remaining commits in temporal order into an 80:20 ratio, allocating the initial 80% as the training set and the subsequent 20% as the testing set. To facilitate learning by mitigating the imbalance observed in the dataset, we applied SMOTE [12] to adjust the ratio of defective commits to clean commits to 1:1 in the training set for each window. As we progressed through sliding windows specific to each project, we measured the values of performance evaluation metrics from the baseline, understandability, and combined models.

3 Results and Discussion

3.1 RQ1: Does the understandability of commits have predictive power for defect-inducing commits?

Figure 4 shows (a) the odd ratios for each feature of the logistic regression model trained with commit understandability features of NeuroJIT and (b) the top 9 feature odd ratios of the logistic regression model trained with combined features. The bold labels on the y-axis indicate that the odd ratios have p-values less than 0.05, signifying that the odd ratios are statistically significant. The black lines passing through each circle represent the 95% confidence intervals of the odd ratios. If a feature's odd ratio is greater than 1, it means that as the feature value increases, the likelihood of a commit being defect-inducing increases; if it is less than 1, the likelihood decreases. Meanwhile, (c) shows the p-values (W_p) from the Wilcoxon rank-sum test, which tests whether defect-inducing commits and clean commits represented by commit understandability features are statistically significantly different [59], and the Cliff's delta effect size (δ), indicating the magnitude of the difference [15]. Both tests are non-parametric and do not require the groups to follow a normal distribution. A $W_p < 0.05$ indicates that the two groups are statistically significantly different and is highlighted in

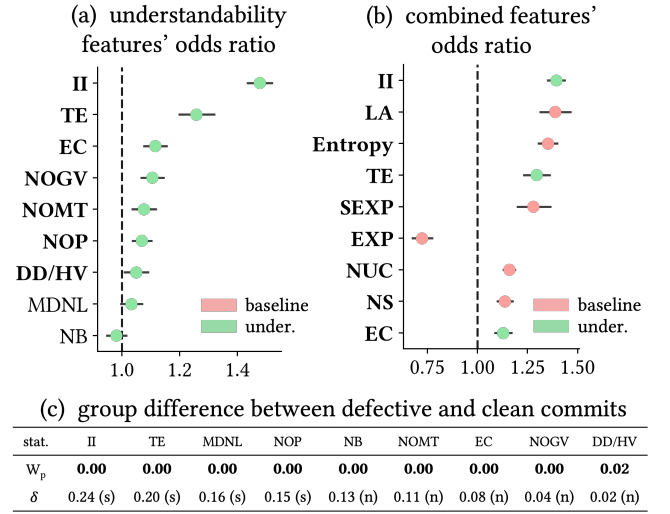


Figure 4: Predictive Power of Understandability Features

bold in the table. The magnitude of differences indicated by δ , are negligible (n) if $\delta \leq 0.147$, small (s) if $0.147 < \delta \leq 0.33$, medium (m) if $0.33 < \delta \leq 0.474$, and large (l) if $\delta > 0.474$.

First, interpreting (a), it is confirmed that most commit understandability features (except for MDNL and NB) show a statistically significant positive correlation with defect-inducing commits. In particular, II and TE, representing the representational complexity of code in commits, show larger odd ratios compared to other features. This aligns with related research findings that modern developers struggle more with the form of source code than its structural complexity as measured by traditional complexity metrics [3, 4]. EC, which represents the frequency of external library calls within a commit, also showed a strong correlation with defect-inducing risks, aligning with modern developers' perception that higher EC makes program comprehension more difficult [3, 58].

To assess whether these features could demonstrate strong predictive power alongside the baseline features, we examined plot (b). In (b), understandability features with smaller correlations did not appear in the top 9 rankings, but II, TE, and EC still showed high odds ratios comparable to the baseline features. The significant predictive power shown by II, TE, EC even among the widely adopted features suggests that the understandability features provide unique and valuable information for predicting defect-inducing commits. This uniqueness was also supported when the understandability features showed little correlation with the baseline features, leading us to investigate this more thoroughly in RQ2.

Finally, (c) reveals that all understandability features are statistically significant in distinguishing the clean and defective commits. The effect sizes of II, TE, MDNL, and NOP are small, showing relatively larger effect sizes compared to other features. Particularly, unlike in (a), MDNL shows significant explanatory power. Qualitative analysis on MDNL revealed that while MDNL generally has low values in most commits, it shows consistently high values in specific defect-inducing commits. This suggests that certain defect-inducing commits with prominent code characteristics resulting in

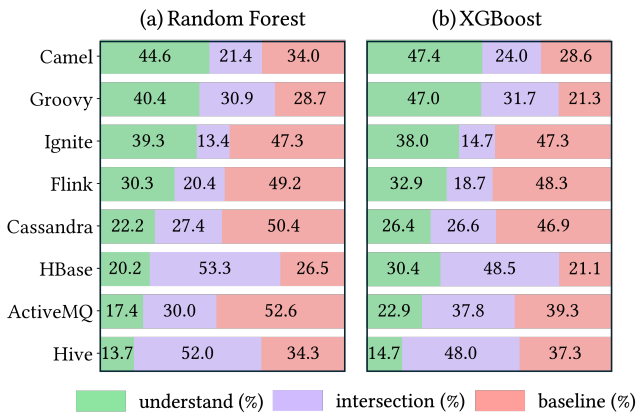


Figure 5: Set Relationships between True Positives Predicted by Understandability Models and Baseline Models

lower understandability can be captured by specific understandability features like MDNL. Consequently, based on the comprehensive interpretation of the plots in Figure 4, we concluded that **the commit understandability features of NeuroJIT have sufficient explanatory power to predict defect-inducing commits.**

3.2 RQ2: Does the understandability of commits provide exclusive information to predict defect-inducing commits?

Figure 5 shows (a) the set relationships between true positives predicted by the random forest model trained on baseline features and the random forest model trained on understandability features, and (b) the set relationships between true positives predicted by the XGBoost model trained on baseline features and the XGBoost model trained on understandability features for each project. The percentages included in the bar graphs represent the proportion of the total true positives. The experimental results indicate that models based on understandability features capture significant amount of defect-inducing commits that the models based on baseline features fail to identify, regardless of the model type. Depending on the project, the random forest models based on understandability features exclusively captured between 13.7% and 44.6%, while the XGBoost models captured between 14.7% and 47.4% of defect-inducing commits. This demonstrates that information on the understandability of commits holds unique insights not possessed by baseline features. We confirmed the uniqueness of the information provided by the understandability features through the true positives that developers are relatively more interested in, and the conclusion remained unchanged even when considering all positives.

To further understand the exclusive information that commit understandability features have about defect-inducing commits, we conducted a qualitative analysis and obtained a comprehensive example that effectively encapsulates the conclusion of the analysis as shown in Figure 6. It illustrates part of commit "cd25880" that changed the "computePlanForTable" method in the Apache HBase project. Models trained on baseline features predicted this commit as non-defective because it changed only one line of code and the

```
project: apache/hbase
commit: cd25880
SimpleRegionNormalizer::computePlanForTable(TableInfo)

182     if (mergeEnabled) {
183         HRegionInfo hri2 = tableRegions.get(candidateIdx+1);
184         long regionSize2 = getRegionSize(hri2);
185         if (regionSize > 0 && regionSize2 > 0 && regionSize +
186             regionSize2 < avgRegionSize) {
187             LOG.info("Table " + table + ", small region size: "
188                 + " plus its neighbor size: " + regionSize2
189                 + ", less than the avg size " + avgRegionSize + ",
190                 merging them");
```

Figure 6: An Example of Exclusive Information Provided by Understandability Features

developers who submitted the commit had extensive experience. However, models trained on understandability features predicted this commit as defect-inducing, mainly because the number of terms used in the context of commit was too high, leading to a high TE and indicating low understandability. This more clearly reveals the unique perspective that understandability features provide in predicting defect-inducing risks, enabling the identification of defect-inducing commits that traditional features fail to detect. Based on the results of Figures 5 and 6, we concluded that **the understandability of commits holds exclusive information for predicting defect-inducing commits**, which is not fully captured by widely adopted just-in-time defect prediction features.

3.3 RQ3: Can the understandability of commits improve the performance of just-in-time defect prediction models?

Figure 7 compares the performance of random forest and XGBoost models trained with baseline features versus those trained with combined features across different projects using radar charts. The radar charts for the five performance metrics are based on the median values of repeated performance validation results for each project. If the project name is displayed in bold, it indicates a statistically significant performance difference between models trained with baseline features and those trained with combined features, with the magnitude of difference indicated by δ . Higher F1, AUC, and MCC values indicate better performance with a larger radar range, while lower Brier and FPR scores indicate better performance with a smaller radar range.

Figure 7 comprehensively illustrates how the combination of understandability features affects the performance of just-in-time defect prediction models. Except for a decrease in the AUC score for the ActiveMQ project, the combination of understandability features resulted in statistically significant performance improvements in specific projects across all evaluation metrics. Improvements in F1, MCC, and AUC were observed in the Groovy, Cassandra, and HBase projects, while improvements based on Brier and FPR were observed in the Camel, ActiveMQ, Ignite, and Cassandra projects. For Flink and Hive, no statistically significant performance differences were found in all evaluation metrics. When significant performance differences occurred, the effect size, measured by the δ , was at least medium. Meanwhile, the trend in performance changes

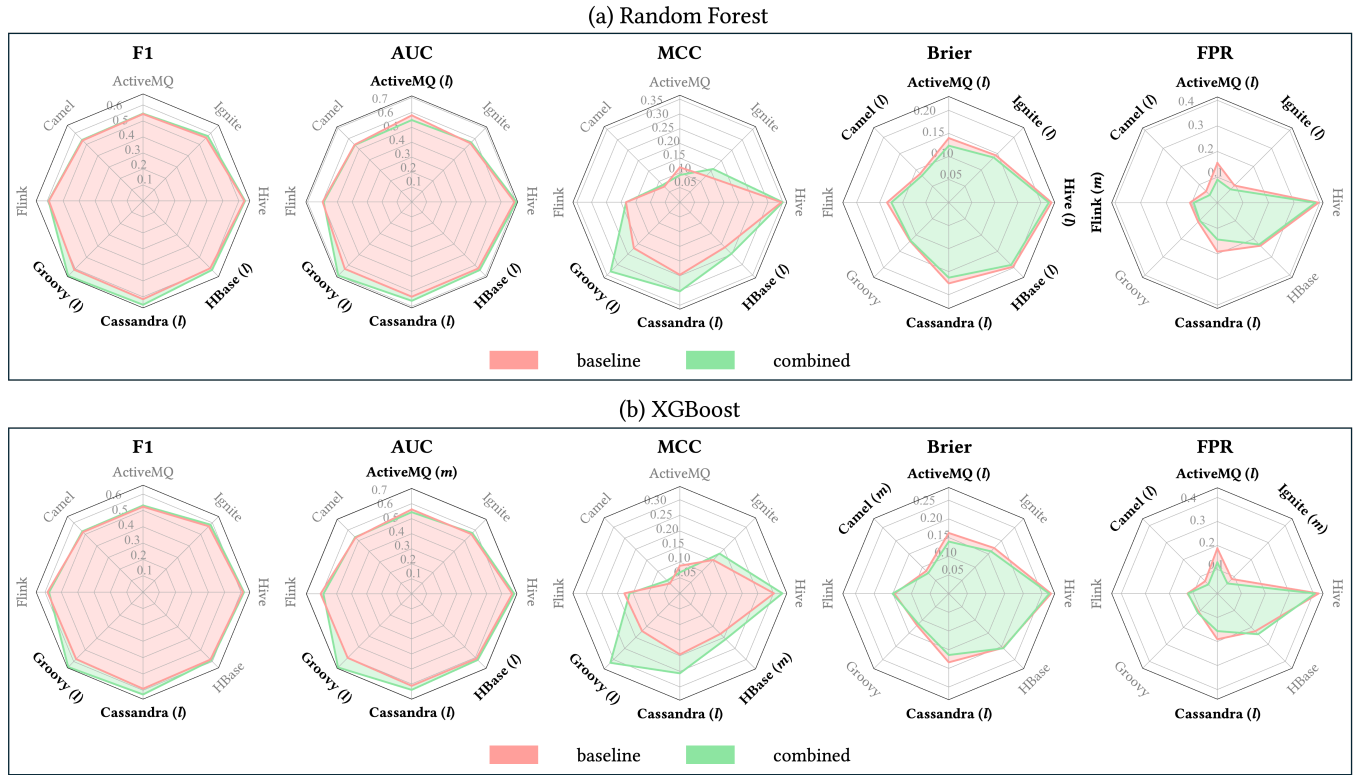


Figure 7: Performance Comparison between Baseline Models and Baseline+Understandability Models

due to the addition of commit understandability features was independent of the algorithm used. Both the random forest model and the XGBoost models exhibited similar changes in evaluation metrics across various projects, with no significant difference in the magnitude of these changes. This indicates that the addition of the understandability features can lead to stable performance improvements that are independent of the algorithm, despite variations across projects and evaluation metrics. Considering the observed results, we concluded that **the combination of understandability features can statistically significantly improve the performance of representative just-in-time defect prediction models trained with widely adopted features.**

3.4 Findings and Limitations

Through the experiments using NeuroJIT, we found that the features of NeuroJIT, (i) correlate with defect-inducing risks; (ii) contain unique information that can predict defect-inducing commits; and (iii) can improve existing just-in-time defect prediction. Our findings provide justification for considering the difficulty in understanding the context of commits and modified code as a significant cause of defect-inducing commits generated by modern developers. Previous just-in-time defect prediction research has primarily focused on information derived from the outcomes of tasks developers perform related to commits, such as those arising from the software development process or source code artifacts. In contrast, our research findings suggest that future related studies should pay

close attention to how developers create commits directly and at which moment commits are created. Based on NeuroJIT, we have opened new avenues by proposing that neuroscience and empirical evaluations could be effective approaches in this regard.

On the other hand, while our study offers a new perspective on just-in-time defect prediction, it also has limitations. First, modern developers use a variety of programming languages, not just Java. Although NeuroJIT's features currently consider only Java, commit understandability issues may vary by language or occur regardless of the language. Therefore, more advanced approaches that either focus on individual languages or transcend languages to fundamentally address developers' understanding of programs could expand our approach. Second, it is essential to more precisely capture the relevant source code that developers need to comprehend when working on commits. By limiting our study to commits that modify methods, we aimed to accurately capture the parts developers need to understand to make required modifications. In fact, we identified that if we were to use commits that modify classes or packages, it could artificially inflate the performance by overemphasizing the scope of what developers need to understand. We believe that there are dynamic or static methods that could more precisely capture the actual context developers need to understand, and that these methods, as part of future evidence-based approaches focusing on commit understandability, will yield better results. We plan to address these limitations in future research and expand NeuroJIT that can provide tangible benefits to developers in practice.

3.5 Toward More Actionable Guidance

Features used in just-in-time defect prediction are crucial because they help developers establish best practices to reduce defect-inducing commits and suggest ways to mitigate these risks [62]. Furthermore, with the growing interest in explainable AI, there has been active discussion on how to leverage explanations for individual predictions to provide actionable guidance to developers in the field of software defect prediction [52]. Given this context, it is important to consider whether just-in-time defect prediction features can offer such actionable guidance. However, the widely adopted features are somewhat distant from the current demand for actionable guidance. For example, baseline features include those representing the number of unique changes to the modified files in the history or the experience of developers who created the commits (i.e., NUC and EXP, which showed strong predictive power in Figure 4 (b)). However, if such features appear in the explanations for individual predictions, developers might struggle to get actionable guidance. This is because developers cannot change the past of commits or alter the experience of the developers involved in the commits.

On the other hand, the understandability features of NeuroJIT encapsulate the characteristics of the modified code and its context within the commits, enabling developers to grasp how to directly modify the source code to improve commit quality. For example, developers can enhance the quality of their commits by recalling the difficulties they experienced during the understanding process through the understandability features. Reviewers can maintain awareness of potential cognitive failures when understanding commits, aiming to identify defect-inducing risks associated with low understandability represented by the features. Therefore, we examined how frequently the understandability features contributed more significantly to individual predictions when combined with baseline features. Considering that developers are likely to base their actions on features with high contribution, if these features appear more frequently in the top feature contribution rankings within each explanation, they could provide more actionable guidance to developers. We used LIME [47], one of the explainable AI techniques, to investigate how often understandability features appear among the top 5 features contributing to prediction results.

Table 3: Average Ratios of Actionable Features within Top 5 Contribution Rankings of LIME Explanations

Project	Random Forest (%)		XGBoost (%)	
	baseline	combined (δ)	baseline	combined
ActiveMQ	54.1	65.4 (<i>l</i>)	59.0	70.7 (<i>l</i>)
Camel	57.4	56.1	56.4	58.2
Flink	61.0	57.2 (<i>s</i>)	38.7	59.3 (<i>l</i>)
Groovy	25.3	54.4 (<i>l</i>)	37.3	43.8 (<i>s</i>)
Cassandra	53.8	68.5 (<i>l</i>)	59.5	82.3 (<i>l</i>)
HBase	52.1	58.4 (<i>s</i>)	49.4	69.9 (<i>l</i>)
Hive	58.2	60.8 (<i>n</i>)	45.7	62.5 (<i>l</i>)
Ignite	53.0	76.8 (<i>l</i>)	47.2	61.9 (<i>m</i>)
Average (%)	51.9	62.2	49.1	63.6

Table 3 shows the ratios of actionable features appearing in the top 5 rankings in the explanations for individual predictions across different models for each project. When understandability features were combined, statistically significant rate changes were highlighted in bold, with the magnitude of the difference represented by δ . The results indicate that explanations from the model trained with combined features can provide statistically more actionable guidance. While there were differences depending on the project, when commit understandability features were combined, they had higher contributions than baseline features in most cases, leading to their much more frequent appearance among the top 5 features. Therefore, we concluded that leveraging commit understandability information in just-in-time defect prediction can not only enhance performance but also provide more actionable guidance.

4 Threats to Validity

4.1 Construct Validity

The experiments in this study were conducted based on ApacheJIT. Consequently, biases that may arise in the process of obtaining commits in ApacheJIT could influence the experiments in this study. Although we performed data preprocessing on the samples and features to mitigate this, it is difficult to ensure that all biases have been eliminated. In the future, we plan to enhance the reliability of this study by utilizing commits from verified open-source projects. Meanwhile, instead of using feature calculators like MetricReloaded [34] and DepDegree [8], we implemented new calculators to obtain commit understandability features. Although we verified the results of our implemented calculators before using them, potential threats arising from this choice still remain. Future studies will be able to validate our verification through the NeuroJIT replication packages that include the calculators.

4.2 Internal Validity

Just-in-time defect prediction datasets inherently contain characteristics such as chronology, concept drift, and verification latency, which could skew experimental results. We employed a sliding window technique that follows the principles used by related studies to address these characteristics while preventing potential biases due to the characteristics. Although we anticipate this approach can mitigate the biases, further verification across more projects with varied durations and domains are needed.

In our study, we used the Spearman correlation coefficient to analyze feature correlations, setting a threshold of 0.7 to denote strong correlations. Thus, correlation interpretations may vary based on the chosen threshold. Nonetheless, even after exploring various thresholds and risking the additional feature removal, we observed no significant changes that could alter the conclusions. We included the correlation analysis scripts in our replication package to facilitate the verification for the selection of thresholds.

The commit understandability features are mostly based on the results of state-of-the-art NeuroSE research and large-scale empirical surveys. Therefore, the reliability of the features in this study may be influenced by the reliability of these research results. To mitigate this threat, we established rigorous criteria for collecting studies and excluded conflicting research results, but the emergence of more related research findings in the future could affect

the experimental results of this study. Additionally, we used the average value as the representative value of commit understandability features extracted from each method. Because the choice of the representative value can affect the experimental results, we also used sum and median values as alternative representative values but did not observe any impact significant enough to change our experimental results.

4.3 External Validity

The experiments in this study were conducted using commits from 8 Apache Java projects, so we cannot guarantee that the same results can be observed in other projects or datasets. Although ApacheJIT has been utilized and verified in the papers of recent premier conferences and major SE journals, we still plan to verify our results on more datasets that fully include the source code of commits and expand the commit understandability features we initially compiled. Meanwhile, a recent study on defect life cycles reported that defects frequently occur in the early stages of projects, suggesting that just-in-time defect prediction models could be built using information from that period alone [48]. However, in our dataset, defects occurred at various stages depending on the project, making the application of that methodology impossible. We believe the findings of this study should be validated in the future when more representative and reasonable methods for just-in-time defect prediction are developed, beyond the sliding window we used.

We limited the scope of this study to commits that modify methods to investigate the relationship between commit understandability and defect-inducing risks. Therefore, it is uncertain whether the same results would be observed for relatively large commits that involve changes related to classes or packages. To improve the generalizability of this study, We plan to conduct follow-up research to automatically and precisely determine the scope of source code that developers need to understand within commits by tracking the runtime contexts of the modified code or the related editing activities within the IDE, and use the results.

We chose random forest and XGBoost based on their strong performance and representativeness in just-in-time defect prediction research [62]. However, they do not represent all machine learning and deep learning algorithms, so the choice of algorithms may affect our findings. In future research, we plan to revisit this study if algorithms with better performance or representativeness emerge in the field of just-in-time defect prediction.

5 Related Works

For just-in-time defect prediction, related studies have focused on "what developers did" and utilized the information within the commits produced by developers. These studies have predicted defect-inducing commits based on direct information such as the diffusion, size, purpose, history, and messages of commits, as well as indirect information related to commits such as developers' experience, issue reports, and change requests. However, there has been limited study focused on "how developers did it" to predict defect-inducing commits based on neurophysiological and empirical evidence representing how developers understood the commits.

Kamei et al. [29] were the first to propose features containing information such as the diffusion and size of commits for just-in-time

defect prediction. A recent literature review on just-in-time defect prediction revealed that approximately 80% of surveyed studies used the features proposed by them [2]. Based on the representativeness of their features, we utilized them as baseline features.

Barnett et al. [7] and Hoang et al. [25] predicted defect-inducing commits using commit messages that contained the intention of code changes. They proposed metrics based on the level of detail in the messages or used deep learning algorithms to feed the messages themselves, aiming to utilize the semantics of the messages. Tourani and Adams [56], Tessema and Abebe [54] predicted defect-inducing risks by leveraging the semantics of discussions and change requests related to commits in issue tracking systems. While the information used in the studies may reflect developers' indirect understanding of the source code, they did not consider the commit understandability as a major trigger for defect-inducing risks.

Liu et al. [36] proposed code churn metrics based on the size of added and deleted lines in commits, and Kondo et al. [31] proposed context metrics that count the number of words, keywords, and indentations in the lines above and below the modified code lines. However, these metrics only depend on the size of the modified code and its context within commits and were proposed based on a fragmented perception of developers rather than an approach to commit understandability. Our study is explicitly based on the hypothesis that commit understandability is related to defect-inducing risks based on the neurophysiological and empirical evidence. We collected various understandability features that comprehensively capture the difficulties of understanding commits.

Trautsch et al. [57] predicted defect-inducing commits by applying a static program analysis tool to all source code files modified by commits, resulting in 270 types of warnings. Warnings based on traditional complexity metrics can partially represent information about the understandability of commits. However, these warnings may exaggerate the parts that developers actually need to understand because they include all the source code that may be unrelated to the tasks developers intend to perform [45]. While the scope of the source code that developers need to understand to create commits is inherently ambiguous, we limited the scope of this study to commits that modify methods, thereby measuring commit understandability features for methods' heads and bodies, which are highly likely to be understood by developers.

6 Conclusion

In this study, we presented a novel approach, NeuroJIT that leverages the correlation between modern developers' neurophysiological and empirical reactions to different code segments and their code characteristics to find the features that can capture the commit understandability. Through NeuroJIT, we discovered that the understandability of commits possesses sufficient explanatory power to predict defect-inducing risks and provides exclusive information from the widely adopted features used so far to identify defect-inducing commits. This allows for the improvement of the performance of the representative just-in-time defect prediction models. Moreover, we revealed that NeuroJIT has the potential to provide more actionable guidance regarding defect-inducing commits.

Our findings underscore the need for research interest in the understandability of commits. Developers are likely to increase

defect-inducing risks due to cognitive failures experienced during the submission or review of commits. Therefore, it is essential to periodically predict commits with high defect-inducing risks due to low understandability and conduct understandability improvement activities for these commits in the software development process. Future just-in-time defect prediction research can utilize NeuroJIT through the replication package of this study, facilitating the application of commit understandability features. As a future research direction, we propose exploring ways to better assess commit understandability regardless of programming language and refining how we capture commit context developers need to understand.

Acknowledgments

This research was partly supported by the MSIT (Ministry of Science and ICT), Korea, under the Convergence security core talent training business support program (IITP-2024-RS-2024-00423071) supervised by the IITP (Institute of Information & Communications Technology Planning & Evaluation) and the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (NRF-2023R1A2C1006390). The second author majors in Bio Artificial Intelligence.

References

- [1] Amine Abbad-Andaloussi. 2023. On the relationship between source-code metrics and cognitive load: A systematic tertiary review. *Journal of Systems and Software* 198 (2023), 111619. <https://doi.org/10.1016/j.jss.2023.111619>
- [2] Shulamyt Ajami, Yonatan Woodbridge, and Dror G. Feitelson. 2017. Syntax, predicates, idioms: what really affects code complexity?. In *Proceedings of the 25th International Conference on Program Comprehension (ICPC '17)*. IEEE Press, Buenos Aires, Argentina, 66–76. <https://doi.org/10.1109/ICPC.2017.39>
- [3] Vard Antinyan. 2020. Evaluating Essential and Accidental Code Complexity Triggers by Practitioners' Perception. *IEEE Software* 37, 6 (2020), 86–93. <https://doi.org/10.1109/MS.2020.2976072>
- [4] Vard Antinyan, Mirosław Staron, and Anna Sandberg. 2017. Evaluating code complexity triggers, use of complexity measures and the influence of code complexity on maintenance time. *Empirical Software Engineering* 22, 6 (Dec. 2017), 3057–3087. <https://doi.org/10.1007/s10664-017-9508-2>
- [5] Alberto Bacchelli and Christian Bird. 2013. Expectations, outcomes, and challenges of modern code review. In *Proceedings of the 2013 International Conference on Software Engineering (ICSE '13)*. IEEE Press, San Francisco, CA, USA, 712–721.
- [6] René Riedl Barbara Weber, Thomas Fischer. 2021. Brain and autonomic nervous system activity measurement in software engineering: A systematic literature review. *Journal of Systems and Software* 178 (2021), 110946. <https://doi.org/10.1016/j.jss.2021.110946>
- [7] Jacob G. Barnett, Charles K. Gathuru, Luke S. Soldano, and Shane McIntosh. 2016. The Relationship between Commit Message Detail and Defect Proneness in Java Projects on GitHub. In *Proceedings of the 13th International Conference on Mining Software Repositories (MSR '16)*. Association for Computing Machinery, New York, NY, USA, 496–499. <https://doi.org/10.1145/2901739.2903496> event-place: Austin, Texas.
- [8] Dirk Beyer and Ashgan Fararooy. 2010. A Simple and Effective Measure for Complex Low-Level Dependencies. In *Proceedings of the 2010 IEEE 18th International Conference on Program Comprehension (ICPC '10)*. IEEE Computer Society, USA, 80–83. <https://doi.org/10.1109/ICPC.2010.49>
- [9] David Bowes, Tracy Hall, and David Gray. 2012. Comparing the performance of fault prediction models which report multiple performance measures: recomputing the confusion matrix. In *Proceedings of the 8th International Conference on Predictive Models in Software Engineering (PROMISE '12)*. Association for Computing Machinery, New York, NY, USA, 109–118. <https://doi.org/10.1145/2365324.2365338> event-place: Lund, Sweden.
- [10] Leo Breiman. 2001. Random Forests. *Machine Learning* 45, 1 (Oct. 2001), 5–32. <https://doi.org/10.1023/A:1010933404324>
- [11] George G. Cabral and Leandro L. Minku. 2023. Towards Reliable Online Just-in-Time Software Defect Prediction. *IEEE Transactions on Software Engineering* 49, 3 (2023), 1342–1358. <https://doi.org/10.1109/TSE.2022.3175789>
- [12] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. 2002. SMOTE: synthetic minority over-sampling technique. *J. Artif. Int. Res.* 16, 1 (June 2002), 321–357. Place: El Segundo, CA, USA Publisher: AI Access Foundation.
- [13] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*. Association for Computing Machinery, New York, NY, USA, 785–794. <https://doi.org/10.1145/2939672.2939785> event-place: San Francisco, California, USA.
- [14] Tianqi Chen, Tong He, Michael Benesty, and Vadim Khotilovich. 2019. Package 'xgboost'. *R version 90*, 1–66 (2019), 40. Publisher: The R Foundation Vienna, Austria.
- [15] Norman Cliff. 1993. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological Bulletin* 114, 3 (1993), 494–509. <https://doi.org/10.1037/0033-2909.114.3.494> Publisher: American Psychological Association.
- [16] Jason Cohen, Steven Teleki, and Eric Brown. 2006. *Best kept secrets of peer code review*. Smart Bear Incorporated.
- [17] J.f. Dooley. 2017. *Software Development, Design and Coding: With Patterns, Debugging, Unit Testing, and Refactoring*. Apress. <https://books.google.co.kr/books?id=LGRADwAAQBAJ>
- [18] Felipe Ebert, Fernando Castor, Nicole Novielli, and Alexander Serebrenik. 2021. An exploratory study on confusion in code reviews. *Empirical Software Engineering* 26, 1 (27 Jan 2021), 12. <https://doi.org/10.1007/s10664-020-09909-5>
- [19] Sarah Fakhoury, Yuzhan Ma, Venera Arnaoudova, and Olusola Adesope. 2018. The Effect of Poor Source Code Lexicon and Readability on Developers' Cognitive Load. In *Proceedings of the 26th Conference on Program Comprehension (ICPC '18)*. Association for Computing Machinery, New York, NY, USA, 286–296. <https://doi.org/10.1145/3196321.3196347> event-place: Gothenburg, Sweden.
- [20] Sarah Fakhoury, Devjeet Roy, Yuzhan Ma, Venera Arnaoudova, and Olusola Adesope. 2020. Measuring the impact of lexical and structural inconsistencies on developers' cognitive load during bug localization. *Empirical Software Engineering* 25, 3 (May 2020), 2140–2178. <https://doi.org/10.1007/s10664-019-09751-4>
- [21] Dror G. Feitelson. 2023. From Code Complexity Metrics to Program Comprehension. *Commun. ACM* 66, 5 (April 2023), 52–61. <https://doi.org/10.1145/3546576> Place: New York, NY, USA Publisher: Association for Computing Machinery.
- [22] Lucian José Gonçalves, Kleinner Farias, and Bruno C. da Silva. 2021. Measuring the cognitive load of software developers: An extended Systematic Mapping Study. *Information and Software Technology* 136 (2021), 106563. <https://doi.org/10.1016/j.infsof.2021.106563>
- [23] Maurice H Halstead. 1977. *Elements of Software Science (Operating and programming systems series)*. Elsevier Science Inc.
- [24] Gao Hao, Haytham Hijazi, João Durães, Júlio Medeiros, Ricardo Couceiro, Chan Tong Lam, César Teixeira, João Castelhamo, Miguel Castelo Branco, Paulo Carvalho, and Henrique Madeira. 2023. On the accuracy of code complexity metrics: A neuroscience-based guideline for improvement. *Frontiers in Neuroscience* 16 (2023). <https://doi.org/10.3389/fnins.2022.1065366>
- [25] T. Hoang, H. Khanh Dam, Y. Kamei, D. Lo, and N. Ubayashi. 2019. DeepJIT: An End-to-End Deep Learning Framework for Just-in-Time Defect Prediction. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE Computer Society, Los Alamitos, CA, USA, 34–45. <https://doi.org/10.1109/MSR.2019.00016>
- [26] Fuqun Huang. 2017. Human Error Analysis in Software Engineering. In *Theory and Application on Cognitive Factors and Risk Management*, Fabio De Felice and Antonella Petrillo (Eds.). IntechOpen, Rijeka, Chapter 2. <https://doi.org/10.5772/intechopen.68392>
- [27] Fuqun Huang, Bin Liu, Shihai Wang, and Qiuying Li. 2015. The impact of software process consistency on residual defects. *Journal of Software: Evolution and Process* 27, 9 (2015), 625–646. <https://doi.org/10.1002/smr.1717> _eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/smr.1717>
- [28] Jirayus Jiarapakdee, Chakkrit Tantithamthavorn, and Christoph Treude. 2018. AutoSpearman: Automatically Mitigating Correlated Software Metrics for Interpreting Defect Models. In *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 92–103. <https://doi.org/10.1109/ICSME.2018.00018>
- [29] Yasutaka Kamei, Emad Shihab, Bram Adams, Ahmed E. Hassan, Audris Mockus, Anand Sinha, and Naoyasu Ubayashi. 2013. A large-scale empirical study of just-in-time quality assurance. *IEEE Transactions on Software Engineering* 39, 6 (2013), 757–773. <https://doi.org/10.1109/TSE.2012.70>
- [30] Hossein Keshavarz and Meiyappan Nagappan. 2022. ApacheJIT: A Large Dataset for Just-in-Time Defect Prediction. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR '22)*. Association for Computing Machinery, New York, NY, USA, 191–195. <https://doi.org/10.1145/3524842.3527996> event-place: Pittsburgh, Pennsylvania.
- [31] Masanari Kondo, Daniel M. German, Osamu Mizuno, and Eun-hye Choi. 2020. The impact of context metrics on just-in-time defect prediction. *Empirical Software Engineering* 25, 1 (Jan. 2020), 890–939. <https://doi.org/10.1007/s10664-019-09736-3>
- [32] Gennaro Laudato, Simone Scalabrino, Nicole Novielli, Filippo Lanubile, and Rocco Oliveto. 2023. Predicting Bugs by Monitoring Developers during Task Execution. In *Proceedings of the 45th International Conference on Software Engineering (Melbourne, Victoria, Australia) (ICSE '23)*. IEEE Press, 1110–1122. <https://doi.org/10.1109/ICSE48619.2023.00100>

- [33] Gichan Lee, Hansae Ju, and Scott Uk-Jin Lee. 2024. *A Replication Package for "NeuroJIT: Improving Just-In-Time Defect Prediction Using Neurophysiological and Empirical Perceptions of Modern Developers"*. <https://doi.org/10.5281/zenodo.13744025>
- [34] B Leijdekkers. 2018. Metrics reloaded: automated code metrics plugin for IntelliJ IDEA. <https://github.com/BasLeijdekkers/MetricsReloaded>
- [35] Gernot Armin Liebchen. 2010. *Data cleaning techniques for software engineering data sets*. Ph.D. Dissertation. Brunel University, School of Information Systems, Computing and Mathematics.
- [36] Jinping Liu, Yuming Zhou, Yibiao Yang, Hongmin Lu, and Baowen Xu. 2017. Code churn: a neglected metric in effort-aware just-in-time defect prediction. In *Proceedings of the 11th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '17)*. IEEE Press, 11–19. <https://doi.org/10.1109/ESEM.2017.8> Place: Markham, Ontario, Canada.
- [37] Shane McIntosh and Yasutaka Kamei. 2018. Are Fix-Inducing Changes a Moving Target? A Longitudinal Case Study of Just-in-Time Defect Prediction. In *Proceedings of the 40th International Conference on Software Engineering (ICSE '18)*. Association for Computing Machinery, New York, NY, USA, 560. <https://doi.org/10.1145/3180155.3182514> event-place: Gothenburg, Sweden.
- [38] Júlio Medeiros, Ricardo Couceiro, Gonalo Duarte, Joo Dures, Joo Castelhana, Catarina Duarte, Miguel Castelo-branco, Henrique Madeira, Paulo De Carvalho, and Csar Teixeira. 2021. Can EEG Be Adopted as a Neuroscience Reference for Assessing Software Programmers' Cognitive Load? *Sensors* 21, 7 (2021). <https://doi.org/10.3390/s21072338>
- [39] Roberto Minelli, , Andrea Mocci, and Michele Lanza. 2015. I know what you did last summer: an investigation of how developers spend their time. In *Proceedings of the 2015 IEEE 23rd International Conference on Program Comprehension (ICPC '15)*. IEEE Press, 25–35. Place: Florence, Italy.
- [40] Sebastian C. Mller and Thomas Fritz. 2016. Using (bio)metrics to predict code quality online. In *Proceedings of the 38th International Conference on Software Engineering (Austin, Texas) (ICSE '16)*. Association for Computing Machinery, New York, NY, USA, 452–463. <https://doi.org/10.1145/2884781.2884803>
- [41] Luca Pascarella, Fabio Palomba, and Alberto Bacchelli. 2019. Fine-grained just-in-time defect prediction. *Journal of Systems and Software* 150 (2019), 22–36. <https://doi.org/10.1016/j.jss.2018.12.001>
- [42] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
- [43] Norman Peitek, Sven Apel, Chris Parnin, Andr Brechmann, and Janet Siegmund. 2021. Program Comprehension and Code Complexity Metrics: An fMRI Study. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. 524–536. <https://doi.org/10.1109/ICSE43902.2021.00056>
- [44] Norman Peitek, Janet Siegmund, Sven Apel, Christian Kstner, Chris Parnin, Anja Bethmann, Thomas Leich, Gunter Saake, and Andr Brechmann. 2020. A Look into Programmers' Heads. *IEEE Transactions on Software Engineering* 46, 4 (2020), 442–462. <https://doi.org/10.1109/TSE.2018.2863303>
- [45] Razvan Petrusel and Jan Mendling. 2013. Eye-Tracking the Factors of Process Model Comprehension Tasks. In *Advanced Information Systems Engineering*, Camille Salinesi, Moira C. Norrie, and scar Pastor (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 224–239.
- [46] Daryl Posnett, Abram Hindle, and Premkumar Devanbu. 2011. A Simpler Model of Software Readability. In *Proceedings of the 8th Working Conference on Mining Software Repositories (MSR '11)*. Association for Computing Machinery, New York, NY, USA, 73–82. <https://doi.org/10.1145/1985441.1985454> event-place: Waikiki, Honolulu, HI, USA.
- [47] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "Why Should I Trust You?": Explaining the Predictions of Any Classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*. Association for Computing Machinery, New York, NY, USA, 1135–1144. <https://doi.org/10.1145/2939672.2939778> event-place: San Francisco, California, USA.
- [48] NC Shrikanth, Suvodeep Majumder, and Tim Menzies. 2021. Early life cycle software defect prediction. why? how?. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 448–459.
- [49] Qinbao Song, Yuchen Guo, and Martin Shepperd. 2019. A Comprehensive Investigation of the Role of Imbalanced Learning for Software Defect Prediction. *IEEE Transactions on Software Engineering* 45, 12 (2019), 1253–1269. <https://doi.org/10.1109/TSE.2018.2836442>
- [50] Davide Spadini, Mauricio Aniche, and Alberto Bacchelli. 2018. PyDriller: Python Framework for Mining Software Repositories. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2018)*. Association for Computing Machinery, New York, NY, USA, 908–911. <https://doi.org/10.1145/3236024.3264598> event-place: Lake Buena Vista, FL, USA.
- [51] Chakkrit Tantithamthavorn and Ahmed E. Hassan. 2018. An experience report on defect modelling in practice: pitfalls and challenges. In *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '18)*. Association for Computing Machinery, New York, NY, USA, 286–295. <https://doi.org/10.1145/3183519.3183547> event-place: Gothenburg, Sweden.
- [52] Chakkrit Kla Tantithamthavorn and Jirayus Jirapakdee. 2021. Explainable AI for Software Engineering. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1–2. <https://doi.org/10.1109/ASE51524.2021.9678580>
- [53] Yida Tao, Yingnong Dang, Tao Xie, Dongmei Zhang, and Sunghun Kim. 2012. How do software engineers understand code changes? an exploratory study in industry. In *Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering (FSE '12)*. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/2393596.2393656> event-place: Cary, North Carolina.
- [54] Hailemeleket Demtse Tessema and Surafel Lemma Abebe. 2021. Enhancing Just-in-Time Defect Prediction Using Change Request-based Metrics. In *2021 IEEE International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. 511–515. <https://doi.org/10.1109/SANER50967.2021.00056>
- [55] Christopher Thunes. 1970. javalang, a lexer and parser targeting Java 8. <https://github.com/c2nes/javalang>
- [56] Parastou Tourani and Bram Adams. 2016. The Impact of Human Discussions on Just-in-Time Quality Assurance: An Empirical Study on OpenStack and Eclipse. In *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, Vol. 1. 189–200. <https://doi.org/10.1109/SANER.2016.113>
- [57] Alexander Trautsch, Steffen Herbold, and Jens Grabowski. 2020. Static source code metrics and static analysis warnings for fine-grained just-in-time defect prediction. In *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 127–138. <https://doi.org/10.1109/ICSME46990.2020.00022>
- [58] Zhiyuan Wan, Xin Xia, Ahmed E. Hassan, David Lo, Jianwei Yin, and Xiaohu Yang. 2020. Perceptions, Expectations, and Challenges in Defect Prediction. *IEEE Transactions on Software Engineering* 46, 11 (2020), 1241–1266. <https://doi.org/10.1109/TSE.2018.2877678>
- [59] R. F. Woolson. 2008. Wilcoxon Signed-Rank Test. In *Wiley Encyclopedia of Clinical Trials*. John Wiley & Sons, Ltd, 1–3. <https://doi.org/10.1002/9780471462422.eoct979> _eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/9780471462422.eoct979>.
- [60] Xin Xia, Lingfeng Bao, David Lo, Zhenchang Xing, Ahmed E. Hassan, and Shanying Li. 2018. Measuring Program Comprehension: A Large-Scale Field Study with Professionals. *IEEE Transactions on Software Engineering* 44, 10 (2018), 951–976. <https://doi.org/10.1109/TSE.2017.2734091>
- [61] Feng Zhang, Quan Zheng, Ying Zou, and Ahmed E. Hassan. 2016. Cross-Project Defect Prediction Using a Connectivity-Based Unsupervised Classifier. In *2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE)*. 309–320. <https://doi.org/10.1145/2884781.2884839>
- [62] Yunhua Zhao, Kostadin Damevski, and Hui Chen. 2023. A Systematic Survey of Just-in-Time Software Defect Prediction. *ACM Comput. Surv.* 55, 10 (Feb. 2023). <https://doi.org/10.1145/3567550> Place: New York, NY, USA Publisher: Association for Computing Machinery.